

仕様署名

三分一信之（三分一技術士事務所）

ver 2.0, 2026-05-22:

Table of Contents

1. Part 1. 開発環境の構築手順	1
1.1. 対象読者	1
1.2. 基本構成	1
1.3. 必要なソフトウェア	1
1.4. ローカル Apache 環境	2
1.5. Linux / EC2 環境	2
1.6. データディレクトリの保護	2
1.7. 動作確認	3
Part 2. データ構造、ファイル配置、プログラム構成	4
対象読者	4
保存データの基本方針	4
ディレクトリ構成	4
ノート JSON	5
主なノード種別	5
プログラム配置	6
主要処理の流れ	6
アップロード	6
ノートへの追加	7
保存	7
読み込み	7
Export / Import	7
保守・機能追加時の注意	7
Part 3. 利用者ガイド	8
ツールの目的と使い方	8
ノートの書き方	8
wilt / bloom / hide の使い方	8
simulation モードの使い方	8
映像の解析方法	9
文書の解析方法	9
プログラム操作方法	9
ホーム画面	9
ノート画面	10
ページ操作	10
コンテキストメニュー	10
保存と読み込み	11
作成したノートの共有方法	11
ノートファイルのダウンロード	11
ノートファイルのアップロード	11

公開	11
共有時の注意	11
2. 付録. 詳細資料	12

Chapter 1. Part 1. 開発環境の構築手順

1.1. 対象読者

この章は、WuWei を設置し、Web サーバー、CGI、データディレクトリ、外部コマンドを管理するシステム管理者を対象にします。

開発者がローカル環境で動作確認を行う場合にも、この章の考え方を使います。

1.2. 基本構成

WuWei は、ブラウザで動作するフロントエンドと、ファイル保存・検索・変換を行う CGI スクリプトから構成されます。

```
wu_wei2/
  index.html
  app2/      ブラウザ側 JavaScript / CSS
  lib/       d3, pdf.js などの外部ライブラリ
  cgi-bin/   ローカル Apache 用 Python CGI
  server/    Linux サーバー用 CGI / 補助コマンド
  data/      利用者データ。GitHub には含めない
  public/    公開ノート。GitHub には含めない
  manual/    マニュアル
```

GitHub で管理するのはプログラム、静的ファイル、マニュアルです。利用者データ、ログ、環境ファイル、アップロードされた実ファイルは GitHub に登録しません。

1.3. 必要なソフトウェア

最低限必要なものは次の通りです。

分類	用途
Web サーバー	Apache または nginx。静的ファイル配信と CGI 実行に使います。
CGI 実行環境	ローカルでは Python CGI、Linux サーバーでは Bash CGI / Python 補助処理を使います。
Python 3	ローカル CGI、アップロード処理、ノート保存処理に使います。
ImageMagick / Poppler	PDF や画像からサムネイルを生成します。
LibreOffice	Office 文書から preview.pdf を生成します。
ffmpeg / ffprobe	動画サムネイル、動画長、動画情報の取得に使います。
zip / unzip	ノートの export / import に使います。

1.4. ローカル Apache 環境

ローカル開発環境では、`C:\Apache24\htdocs\wu_wei2` のように Apache の公開ディレクトリへ配置します。

確認項目は次の通りです。

- `index.html` がブラウザから開ける。
- `cgi-bin/*.py` が CGI として実行できる。
- `data/` に書き込みできる。
- `app2/` と `lib/` の静的ファイルが読み込める。
- PDF、画像、Office、動画を扱う場合は外部コマンドのパスが通っている。

ローカル環境では、通常 `guest` 利用者として動作確認できます。

1.5. Linux / EC2 環境

Linux サーバーでは、`/var/www/wu_wei2` のような配置先に GitHub から clone します。

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/pontsoleil/wu_wei.git wu_wei2
sudo chown -R ec2-user:ec2-user /var/www/wu_wei2
```

利用者データとログは、GitHub には含めずサーバー上で作成します。

```
cd /var/www/wu_wei2
mkdir -p data
mkdir -p public/note
mkdir -p server/log
```

Web サーバー実行ユーザーが書き込めるようにします。

```
sudo chown -R ec2-user:nginx data public server/log
chmod -R 775 data public server/log
chmod +x server/*.cgi server/is-login server/bin/*
```

実際の nginx / fcgiwrap 設定は [manual/AWS/setup_on_EC2.adoc](#) を参照してください。

1.6. データディレクトリの保護

`data/` 配下には、アップロードファイル、サムネイル、プレビュー、ノート本文が保存されます。

ブラウザから `/wu_wei2/data/...` に直接アクセスできないようにします。ファイル参照は `load-file.cgi`

を經由させ、ログイン状態、利用者 ID、相対パスの検証を行います。

基本方針は次の通りです。

- `data/` は直接公開しない。
- JSON には OS 絶対パスを保存しない。
- JSON には `area`、`path`、`role` を保存する。
- 実ファイルは CGI が検証後に返す。

1.7. 動作確認

管理者は、導入後に次を確認します。

- ホーム画面が表示される。
- ファイルをアップロードできる。
- アップロード後にサムネイルが表示される。
- Office 文書の場合、`preview.pdf` が生成される。
- ノートを保存し、再読み込みできる。
- ノートを `export / import` できる。
- 公開機能を使う場合、`public/note` に公開ノートが作成される。

Part 2. データ構造、ファイル配置、プログラム構成

対象読者

この章は、WuWei の保守、デバッグ、機能追加を行う開発者を対象にします。

画面上の操作と保存データ、描画用 runtime graph、CGI の責務の対応を理解することが目的です。

保存データの基本方針

WuWei の保存データは、次の分離を基本にします。

種類	主な場所	役割
アップロード実 体	<code>data/{user_id}/upload/YYYY/MM/DD/{upload_uuid}/</code>	元ファイル、サムネイル、Office 変換 PDF、動画情報などを保存します。
リソース定義	<code>data/{user_id}/resource/YYYY/MM/DD/{resource_uuid}/resource.json</code>	ホーム画面、検索、ノートへの追加で使うメタデータです。
ノート	<code>data/{user_id}/note/YYYY/MM/DD/{note_uuid}/note.json</code>	ノート JSON を UTF-8 JSON として保存します。ノード、リンク、グループ、ページを保持します。
公開ノート	<code>public/note/...</code>	公開閲覧用に生成されたノートです。

重要なのは、ファイル本体の正本は `upload/`、ノート構造の正本は `note.json` である、という分離です。

ディレクトリ構成

```
data/{user_id}/
  upload/
    YYYY/MM/DD/{upload_uuid}/
      manifest.json
      {original_file}
      thumbnail.jpg
      preview.pdf

  resource/
    YYYY/MM/DD/{resource_uuid}/
      resource.json

  note/
    YYYY/MM/DD/{note_uuid}/
      note.json

  trash/
```

通常保存では、アップロードファイルをノートディレクトリへコピーしません。ノート JSON には参照を保存します。これにより、大きなファイルの重複コピーを避けます。

ノート JSON

ノートは複数の Page を持ちます。各 Page は、画面上の作業面です。

```
{
  "note_id": "_note_uuid",
  "note_name": "note name",
  "description": "",
  "currentPage": "_page_uuid",
  "pages": [
    {
      "id": "_page_uuid",
      "pp": 1,
      "name": "",
      "description": "",
      "nodes": [],
      "links": [],
      "groups": [],
      "transform": {
        "x": 0,
        "y": 0,
        "scale": 1
      },
      "thumbnail": "<svg ...>"
    }
  ]
}
```

`nodes[]`、`links[]`、`groups[]` が WuWei のノート本体です。

- `nodes[]`: Content、Topic、Memo、Segment、PageMarker など。
- `links[]`: ノード間の関係。
- `groups[]`: 複数ノードのまとまり。Timeline axis、Contents axis も group として保存します。

`common.current.page` は保存対象となる正本です。`common.graph` は `current.page` から展開される描画用 runtime graph であり、pseudo node / pseudo link を含むことがあります。保存時に `common.graph` から `current.page` へ逆変換する専用処理は行いません。実ノードや実リンクは原則として `current.page` 側の同一オブジェクト参照として更新され、pseudo 要素は保存対象に含めません。

主なノード種別

種別	役割
Content	読解対象となる文書、映像、Web ページ、画像、アップロードファイルを表します。

種別	役割
Topic	論点、概念、キーワード、人物、出来事、仮説などを表します。
Memo	自由記述の付箋です。気づき、補足、疑問、作業メモを書きます。
Segment	動画または音声の時間区間を表します。Timeline axis 上に配置します。
PageMarker	PDF や Office preview PDF の特定ページを表します。Contents axis 上に配置します。
Link	資料、論点、メモ、ページ、映像区間の関係を表します。
Group	複数ノードのまとめ、または Timeline / Contents の軸定義を表します。

プログラム配置

場所	役割
app2/	ブラウザ側アプリケーション本体です。
app2/model/	ノート、ノード、リンク、リソース、動画、Timeline、Contents などのモデル処理を持ちます。
app2/draw/	SVG キャンバスへの描画、再描画、simulation モードの描画更新を担当します。
app2/menu/	メニュー、コンテキストメニュー、ページ一覧、アップロード、公開などの UI 操作を担当します。
app2/info/	Content、動画、アップロードファイル、Timeline などの情報表示を担当します。
app2/edit/	ノードやリソースの編集フォームを担当します。
app2/home/	保存済みノートと Resource library の入口画面を担当します。
app2/misc/	共通状態、ユーティリティ、Ajax、SSE などを持ちます。
cgi-bin/	ローカル Apache 用の Python CGI です。
server/	Linux サーバー用 CGI と補助コマンドです。
manual/	マニュアルです。

主要処理の流れ

アップロード

1. 利用者がファイルを選択します。
2. サーバーが [upload/YYYY/MM/DD/{upload_uuid}/](#) に元ファイルを保存します。
3. PDF / 画像 / Office / 動画に応じてサムネイルや preview を生成します。
4. Resource library 用の [resource.json](#) を作成します。upload bundle の [manifest.json](#) は [resource.json](#) の [storage.manifest](#) から参照します。
5. ホーム画面や検索画面で利用できるようになります。

ノートへの追加

1. ホーム画面で Resource を選択します。
2. 現在のノートに Content ノードを追加します。
3. Content ノードはファイル本体を持たず、Resource 情報と storage 参照を持ちます。
4. 必要に応じて Topic、Memo、Link、Group を追加します。

保存

1. 保存正本である `current.page` を含むノート JSON を確認します。
2. `pseudo node` / `pseudo link` などの runtime 要素を保存対象に含めません。
3. ノート JSON を UTF-8 JSON として書き出します。
4. `note/YYYY/MM/DD/{note_uuid}/note.json` に保存します。

読み込み

1. `note.json` を読み込みます。
2. `pages[]` と `currentPage` から現在ページを決定します。
3. `current.page` から `common.graph` を再構築し、描画します。

Export / Import

Export は、保存済みノート JSON が参照する `upload/` ディレクトリを収集して ZIP 化します。

Import は、ZIP 内のノート JSON と upload resource を復元し、現在利用者の領域に保存します。

保守・機能追加時の注意

- `data/` 配下を直接 URL 公開しない。
- 保存 JSON に OS 絶対パスを保存しない。
- `current.page` を保存正本、`common.graph` を runtime 展開として扱う。
- 表示用 `pseudo node` / `pseudo link` を保存 JSON の正本に混ぜない。
- Page の `thumbnail` はページ一覧やノート一覧の表示に使われるため、描画キャンバスのクリア処理で巻き込まない。
- 新しいノード種別を追加する場合は、モデル、描画、編集、保存、読み込み、export/import を一通り確認する。
- Office、PDF、動画など外部コマンドに依存する機能は、導入環境で degrade できるようにする。

Part 3. 利用者ガイド

ツールの目的と使い方

WuWei は、資料を単に保存するためのツールではありません。

PDF、Office 文書、Web ページ、画像、動画、音声などを Content として置き、それをどのように読み、どの観点で重要だと考えたかを Topic、Memo、Link として残すための調査ノート環境です。

WuWei で作るノートは、完成した結論だけではなく、調査途中の仮説、気づき、比較、関連づけ、あとで見直したい未解決事項も扱います。

ノートの書き方

基本の流れは次の通りです。

1. 読む対象を Content として登録します。
2. 重要な語句、論点、仮説を Topic として置きます。
3. 気づきや補足を Memo として書きます。
4. Content、Topic、Memo を Link で結びます。
5. 関係が増えたら Group や Page で整理します。
6. 必要な範囲だけ表示し、見直しながらノートを育てます。

Content は資料そのものを表します。Topic は「その資料をどの観点で読むか」を表します。Memo は自由なコメントです。Link は「なぜ関係があるのか」を残すための線です。

wilt / bloom / hide の使い方

調査が広がると、画面上のノードとリンクが増えます。WuWei では表示範囲を調整できます。

操作	意味
wilt	起点ノードからリンクをたどって、関連範囲を折りたたみます。いま注目しない周辺を隠すために使います。
bloom	非表示になっているリンクと、その先のノードを再表示します。wilt と違い、通常は 1 段ずつ開きます。
hide	選択した Node、Link、Group を直接非表示にします。削除ではありません。

hide した孤立ノードは、リンクを経由した **bloom** では戻せない場合があります。あとで戻したい要素は、関連リンクを残してから折りたたむと扱いやすくなります。

simulation モードの使い方

simulation モードは、ノード同士の関係を力学モデルで動かし、配置を調整する表示モードです。

主な用途は次の通りです。

- ノードが密集して見づらいときに、関係に応じて散らす。
- Link の多い中心的な Topic を見つける。
- Group のまとまりを確認する。
- 手作業で配置する前の下書きとして使う。

simulation モードでは、配置が動くため、最終的な見せ方を整えるときは通常モードで位置を調整します。

映像の解析方法

動画や音声は、全体を Content として置き、重要な時間区間を Segment として切り出します。

基本の流れは次の通りです。

1. 動画をアップロード、または動画 URL を Content として追加します。
2. 動画プレイヤーで重要な場面を探します。
3. 開始時刻と終了時刻を指定して Segment を作成します。
4. Segment に、その場面の論点を表す Topic をリンクします。
5. 必要に応じて Memo に発言、観察、疑問を書きます。

Segment を使うと、動画全体ではなく「どの場面がどの論点に関係するか」を記録できます。

文書の解析方法

PDF や Office 文書は、全体を Content として置き、重要ページを PageMarker として扱います。

基本の流れは次の通りです。

1. 文書をアップロードします。
2. Office 文書の場合は preview PDF が生成されます。
3. 文書 Content から Contents axis を作成します。
4. 重要ページを PageMarker として追加します。
5. PageMarker から Topic や Memo にリンクします。

PageMarker を開くと、対象文書の該当ページを表示します。文書のページと論点を分けて記録することで、後から根拠をたどりやすくなります。

プログラム操作方法

ホーム画面

ホーム画面では、保存済みノートと登録済み Resource を探します。

主な操作は次の通りです。

- 保存済みノートを開く。
- Resource library から文書、画像、動画などを探す。
- Resource を現在のノートに追加する。
- Resource から新規ノートを作る。
- ファイルをアップロードする。

ノート画面

ノート画面では、Content、Topic、Memo、Link、Group を配置します。

主な操作は次の通りです。

- ノードを追加する。
- ノード同士をリンクする。
- ノードを編集する。
- ノードやリンクを移動・削除・非表示にする。
- Page を追加、コピー、削除する。
- ノートを保存する。

ページ操作

ノートは複数ページを持てます。

ページ一覧では、各ページのサムネイル、ページ番号、ページ名を確認できます。

主な操作は次の通りです。

- Page を開く。
- Page 名と説明を編集する。
- Page をコピーする。
- Page を追加する。
- Page を削除する。

コンテキストメニュー

ノード、リンク、Group を選択すると、対象に応じた操作メニューを開けます。

代表的な操作は次の通りです。

- edit
- info
- link / connect
- copy / paste / clone

- wilt / bloom / hide
- group / ungroup
- delete

保存と読み込み

作業中のノートは保存メニューから保存します。

保存すると、ノート名、説明、ページ、ノード、リンク、グループ、サムネイルが保存されます。

保存済みノートはホーム画面またはノート一覧から再度開けます。

作成したノートの共有方法

ノートファイルのダウンロード

ノートメニューからダウンロードすると、現在のノートを持ち運び用のファイルとして出力できます。

ダウンロードファイルには、ノート JSON と、ノートが参照する upload resource が含まれます。

ノートファイルのアップロード

別環境でノートファイルをアップロードすると、ノート JSON と参照先 resource を復元できます。

同じ SHA-256 のアップロードファイルが既に存在する場合は、既存 resource を再利用できます。

公開

公開機能を使うと、ノートを公開 URL で閲覧できるようにします。

公開時は、現在のノートを保存し、その保存済み内容から公開用データを作成します。

公開閲覧モードでは、読者がノートの構造、Content、Topic、Memo、Link、Page をたどれることを優先します。

共有時の注意

- 未保存の変更がある場合は、先に保存します。
- 公開したくないファイルや個人情報が Content / Memo に含まれていないか確認します。
- 外部公開環境では、**data/** を直接公開しない設定が必須です。
- 相手に編集させたい場合は、公開 URL ではなくノートファイルの export / import を使います。

Chapter 2. 付録. 詳細資料

詳細な仕様や手順は、次の文書を参照してください。

文書	内容
manual/setup/setup.adoc	データディレクトリ保護、 load-file.cgi 、サーバー設定の考え方。
manual/AWS/setup_on_EC2.adoc	Amazon Linux 2 / nginx / fcgiwrap での導入手順。
manual/data/data.adoc	保存データ、upload、note、manifest、thumbnail、preview の仕様。
manual/note_resource/role.adoc	ノートとリソースを扱う CGI の役割分担。
manual/home/wuwei_home_spec.adoc	ホーム画面と Resource library の仕様。
manual/user_guide/wuwei_guide.adoc	利用者向けの詳細操作ガイド。